دانشکده مهندسی مکانیک

پردیس دانشکده‌های فنی

دانشگاه تهران

هوش مصنوعی – نیمسال دوم ۴-۱۴۰۳

تمرین ششم

(یادگیری تقویتی)

نگارش:

علی خیری

۸۱۰۶۰۳۰۷۱

استاد درس:

دکتر شریعت‌پناهی

تابستان ۱۴۰۴

فهرست مطالب

۱. محیط با اقدام‌ها و فضای حالت گسسته ۳

۱-۱ آماده‌سازی محیط ۳

۱-۲ الگوریتم یادگیری Q - Q-Learning ۵

۱-۳ الگوریتم یادگیری Q دوگانه – Double Q-Learning ۸

۱-۴ الگوریتم DQN 11

۱-۵ بررسی تاثیر هایپرپارامترها در الگوریتم DQN ۱۵

۲. محیط با اقدام‌های پیوسته ۱۸

۲-۱ الگوریتم DQN. ۱۸

۲-۲ الگوریتم DDPG ۲۱

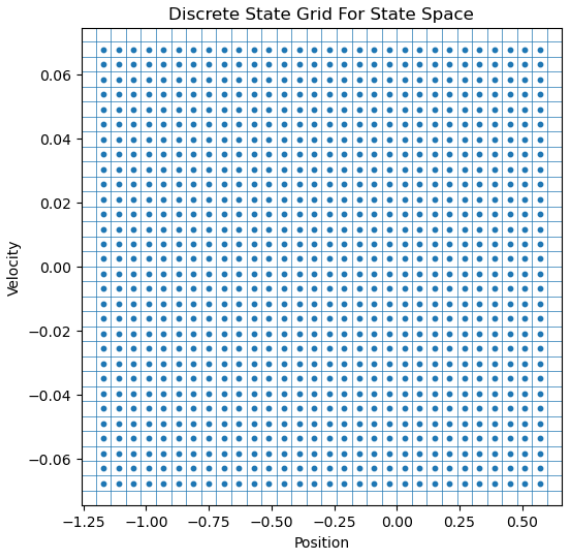
۳. مقایسه مدل‌ها و نتیجه‌گیری ۲۸

۱. محیط با اقدام‌ها و فضای حالت گسسته

هدف محیط mountaincar یافتن استراتژی مناسب برای رساندن خودرو به بالای تپه می‌باشد. این مسئله از نوع Delay-Reward می باشد زیرا برای رسیدن به بالای تپه که در سمت راست بوده، گاهی لازم است خودرو به سمت چپ حرکت کند که در ظاهر به معنای دور شدن از هدف می‌باشد.

۱-۱ آماده‌سازی محیط

در ابتدا نیاز است کلاسی تعریف شده و تمامی خواص محیط از جمله بازه سرعت و موقعیت، معادلات حرکت، شروط توقف و همچنین تابع پاداش در آن تعریف شود. از اساسی ترین خاصیت‌های این محیط این است که بازه فضای حالت (موقعیت و سرعت) به ۳۰ قسمت تقسیم شود. شکل زیر نمونه فضای حالت گسسته را نمایش می‌دهد:



شکل ۱ – فضای حالت گسسته حالت ۱

همچنین از موارد مهم دیگری که باید در کلاس محیط تعریف شود نوع پاداش داده‌شده به Agent می‌باشد. در حالات اولیه این پاداش برابر ۱- برای هر گامی که خودرو به پرچم نرسیده باشد در نظر گرفته می‌شود. بدین ترتیب در یک Episode که شامل ۲۰۰ گام می‌باشد، اگر خودرو به پرچم نرسد پاداش آن برابر ۲۰۰- خواهد بود و هر چه خودرو سریعتر به هدف برسد پاداش آن بیشتر خواهد بود.

برای هر الگوریتمی که پیاده‌سازی شود نتایج زیر ذکر و بررسی خواهد شد:

۱- نمودار پاداش به ازای هر مرحله در طول فرآیند یادگیری

۲- نمودار تعداد گام‌ها به ازای هر مرحله در طول فرآیند یادگیری

۳- نمودار پنجره محرک (moving average) پاداش به ازای هر مرحله در طول فرآیند یادگیری – که در این نمودار میانگین پاداش هر ۱۰۰ مرحله گرفته شده (از نمودار اولی) و نمایش داده می‌شود. این نمودار به لحاظ بصری راحت‌تر تفسیر می‌شود.

۴- زمان آموزش مدل

۵- میانگین پاداش (۱۰۰ اپیزود آخر)

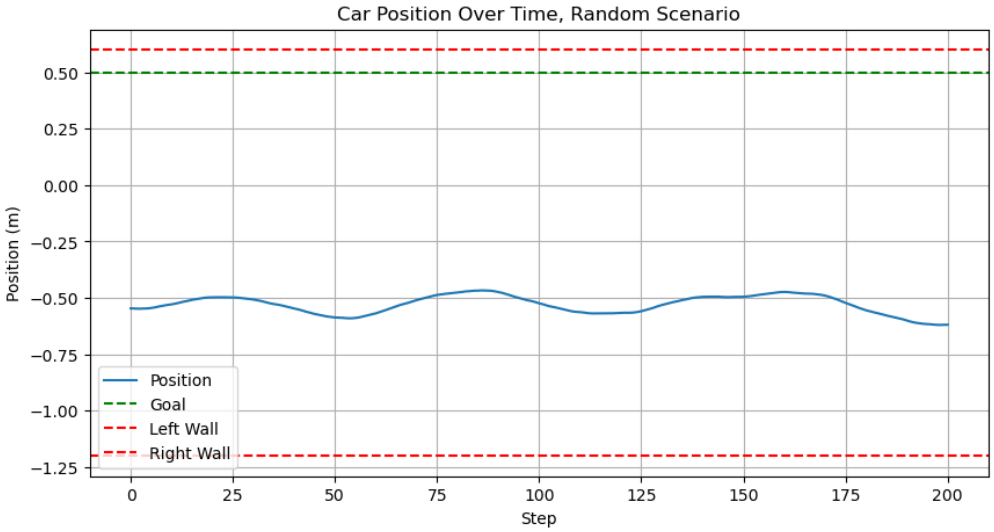
۶- درصد موفقیت (۱۰۰ اپیزود آخر)

۷- تعداد گام‌های طی‌شده (۱۰۰ اپیزود آخر)

۸- فیلم عملکرد اپیزود تست

۹- نمودار جابجایی و سرعت خودرو در اپیزود تست

برای مثال نمودار جابجایی خودرو در یک سناریو اتفاقی (مدل آموزش ندیده) بصورت زیر خواهد بود:



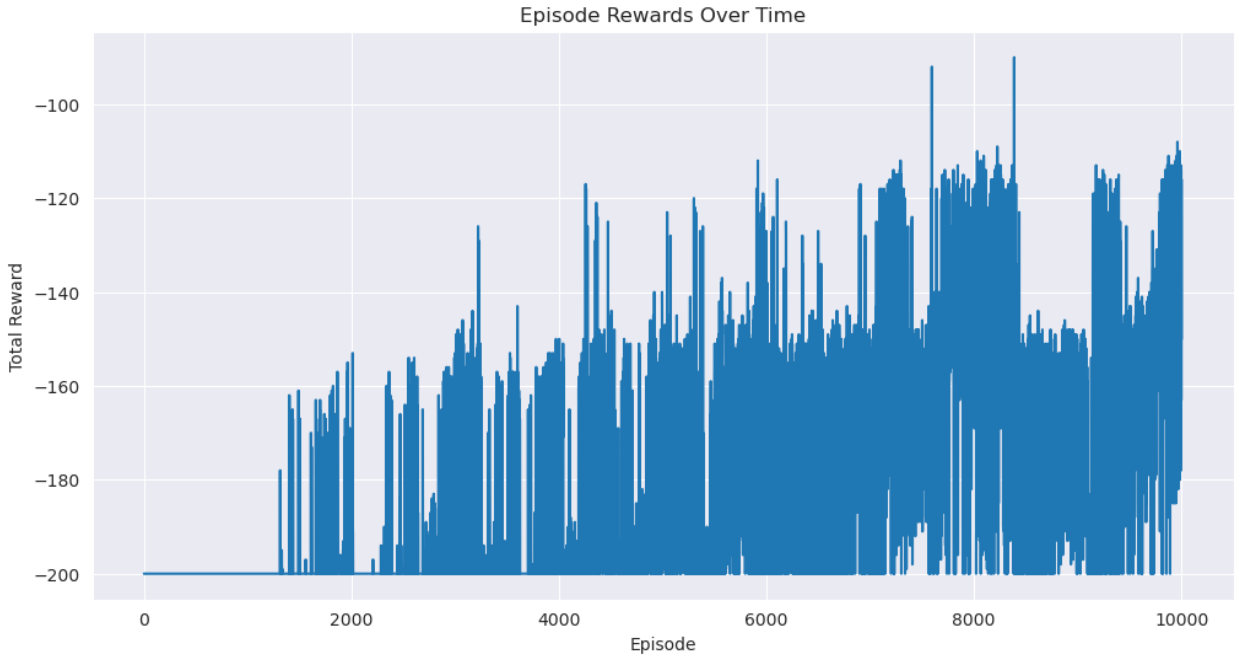
شکل ۲ – موقعیت خودرو برای یک اپیزود مدل آموزش داده‌شده

۱-۲ الگوریتم یادگیری Q - Q-Learning

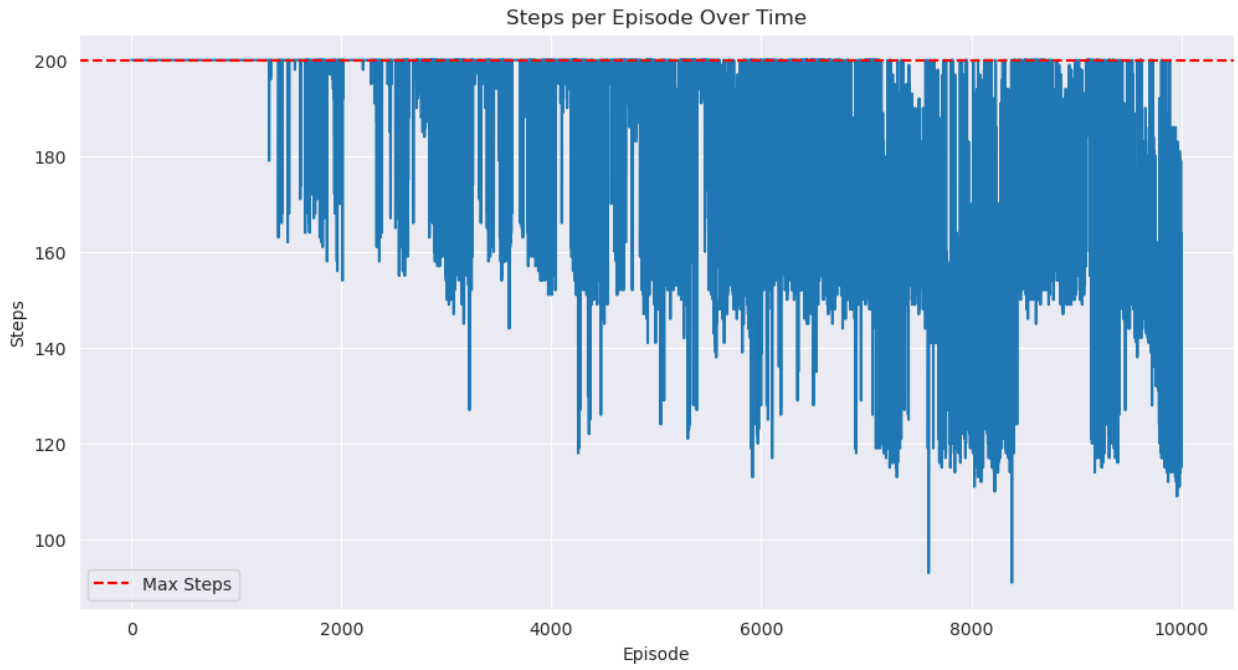
هدف این الگوریتم یافتن بهترین اقدام در هر حالت می‌باشد. بدین منظور یک ماتریس ۳۰ در ۳۰ در ۳ تشکیل شده (۳۰ برای موقعیت، ۳۰ برای سرعت، ۳ برای تعداد اقدام‌ها ممکن) که هر کدام از این درایه‌ها ارزش متناسب با هر اقدام می‌باشد. در طی فرآیند یادگیری ارزش این اقدام ها بر اساس به‌روزرسانی می‌شود.

پس از یادگیری، نتایج این الگوریتم به شرح زیر می‌باشد.

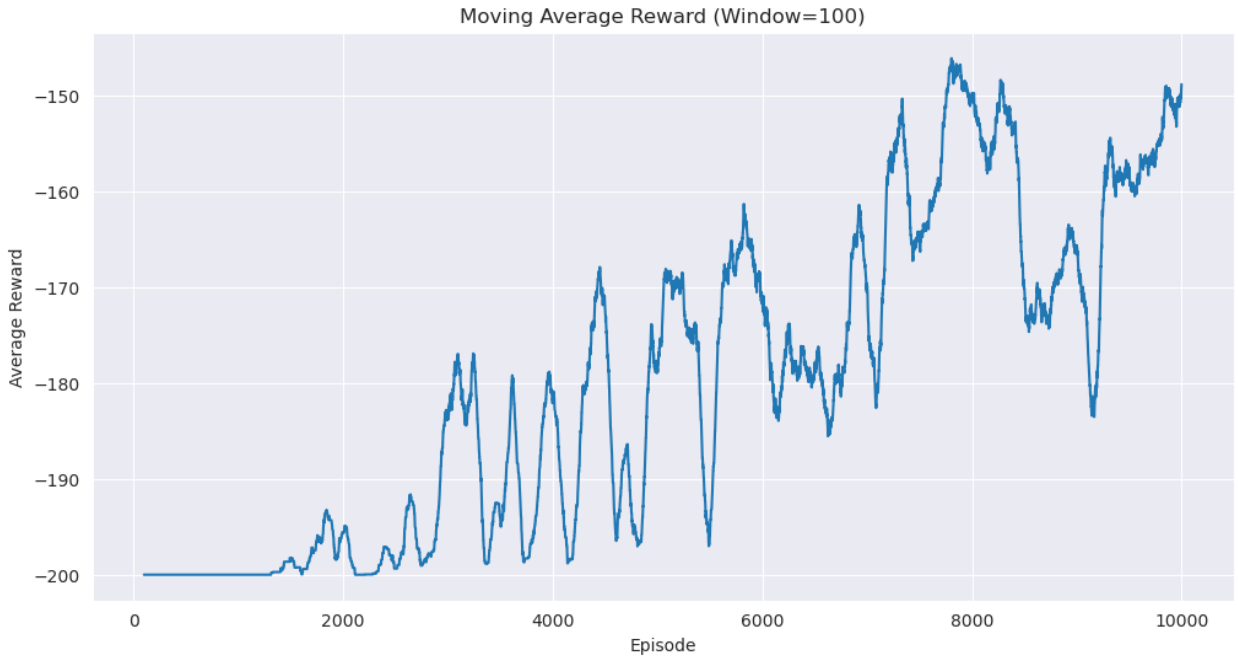
زمان آموزش برحسب ثانیه	۹۲,۴۳
پاداش (۱۰۰ اپیزود آخر)	-۱۴۸
درصد موفقیت (۱۰۰ اپیزود آخر)	۱۰۰٪
تعداد گام‌های طی‌شده (۱۰۰ اپیزود آخر)	۱۴۹



شکل ۳ – نمودار پاداش در طول یادگیری برای الگوریتم یادگیری Q



شکل ۴ – نمودار تعداد گام‌های طی‌شده در طول یادگیری برای الگوریتم یادگیری Q

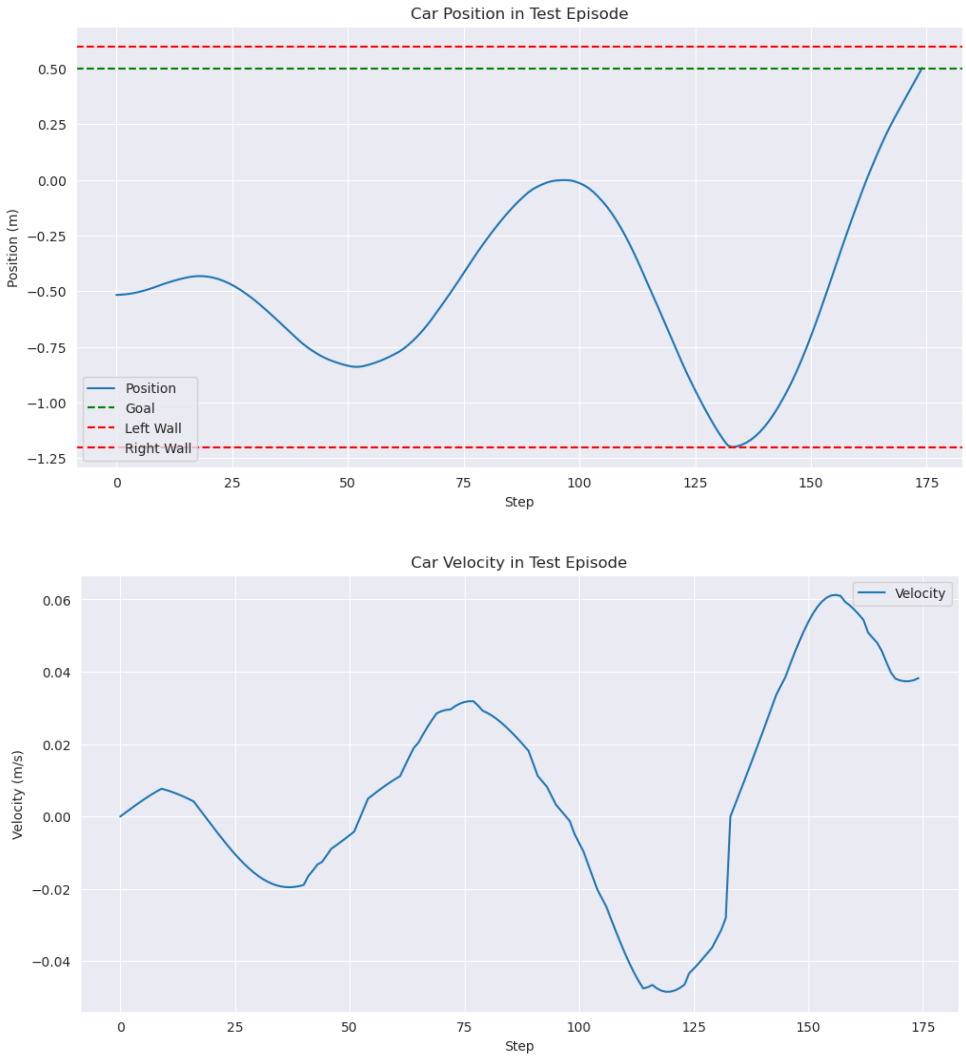


شکل ۵ - نمودار پنجره محرک پاداش به ازای هر مرحله در طول فرآیند یادگیری برای الگوریتم یادگیری Q

همانطور که از نمودارپنجره محرک پاداش مشخص است، در مراحل اول یادگیری (حدود ۲۰۰۰ اپیزود اول) پاداش ۲۰۰- بوده که به این معنی است که خودرو به هدف نمی‌رسد. با گذشت زمان پاداش افزایش یافته و همینطور تعداد گام‌ها کاهش می‌یابد. در ۱۰۰ اپیزود آخر خودرو به طور کامل به هدف می‌رسد که نشان می‌دهد این الگوریتم برای این محیط که شامل فضای حالت و تعداد اقدام‌های محدود بوده بسیار موفق عمل می‌کند.

اپیزود تست این الگوریتم با نام Q-learning در پوشه تست قرار گرفته شده‌است.

نمودارهای موقعیت و سرعت این اپیزود بصورت زیر می‌باشد:



شکل ۶ – نمودار موقعیت و سرعت خودرو در اپیزود تست

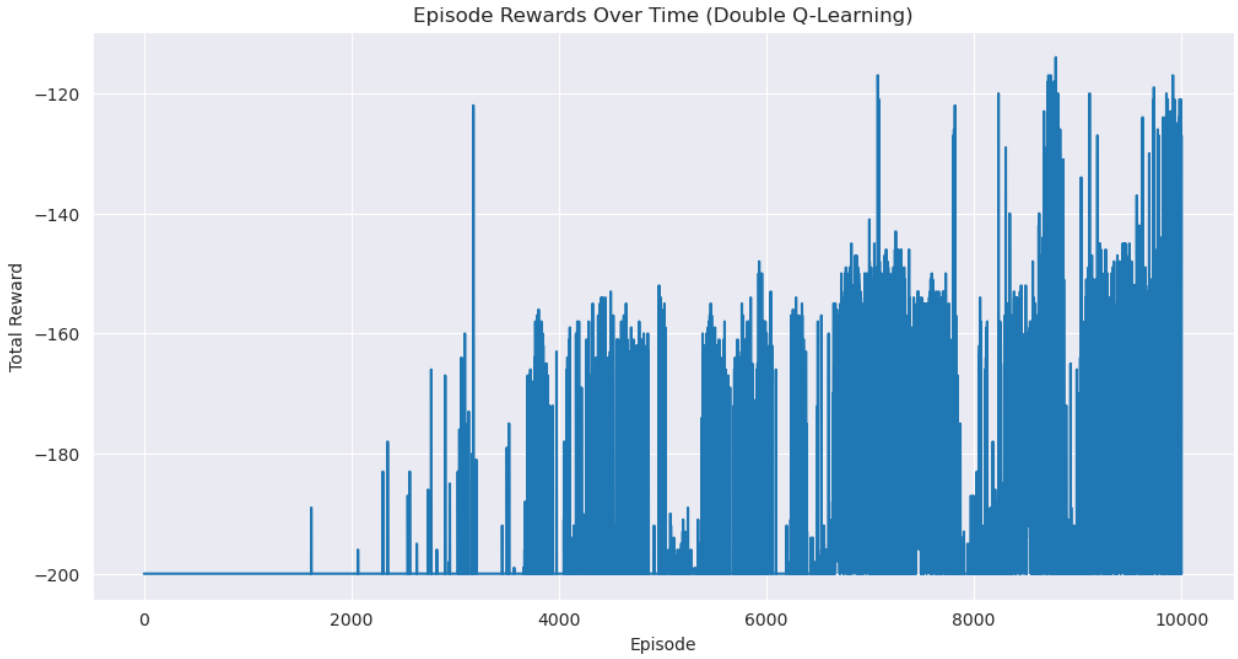
همانطور که مشخص است، خودرو در انتها به هدف (خط سبز رنگ) می‌رسد.

۳-۱ الگوریتم یادگیری Q دوگانه – Double Q-Learning

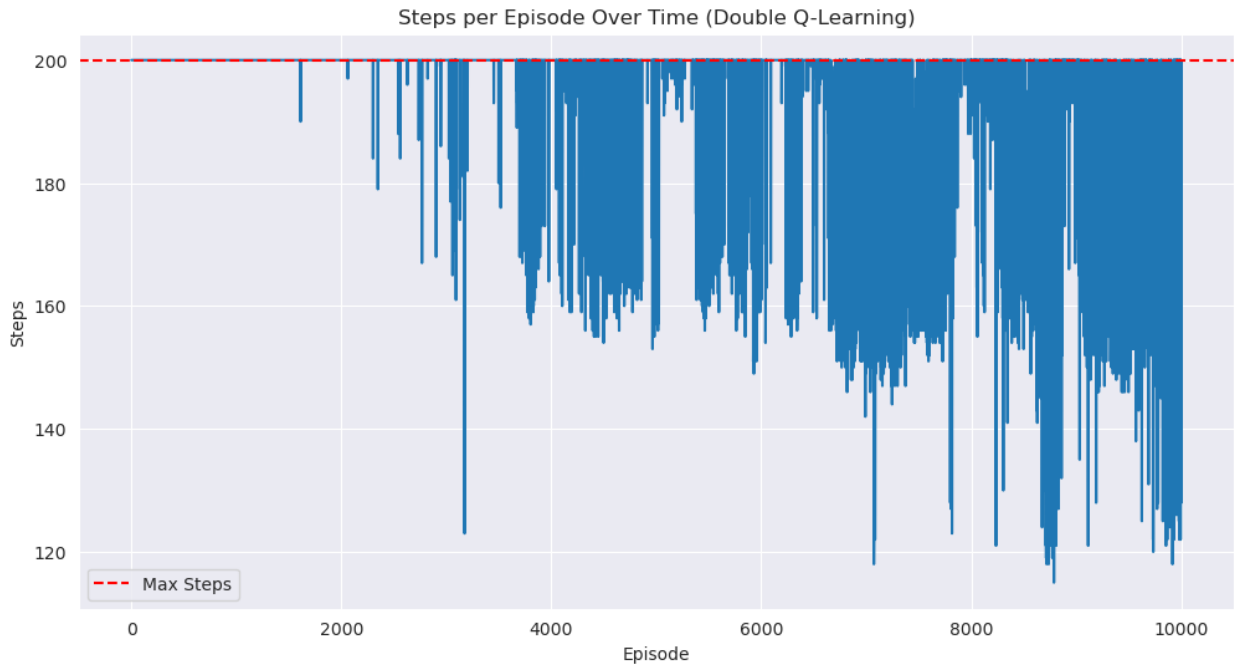
این الگوریتم با جداسازی جداول انتخاب عمل و تخمین ارزش، از مقادیر Q بیش از حد خوش‌بینانه که می‌تواند منجر به سیاست‌های غیربهینه در یادگیری Q استاندارد شود، جلوگیری می‌کند.

نتایج این الگوریتم به‌شرح زیر می‌باشد:

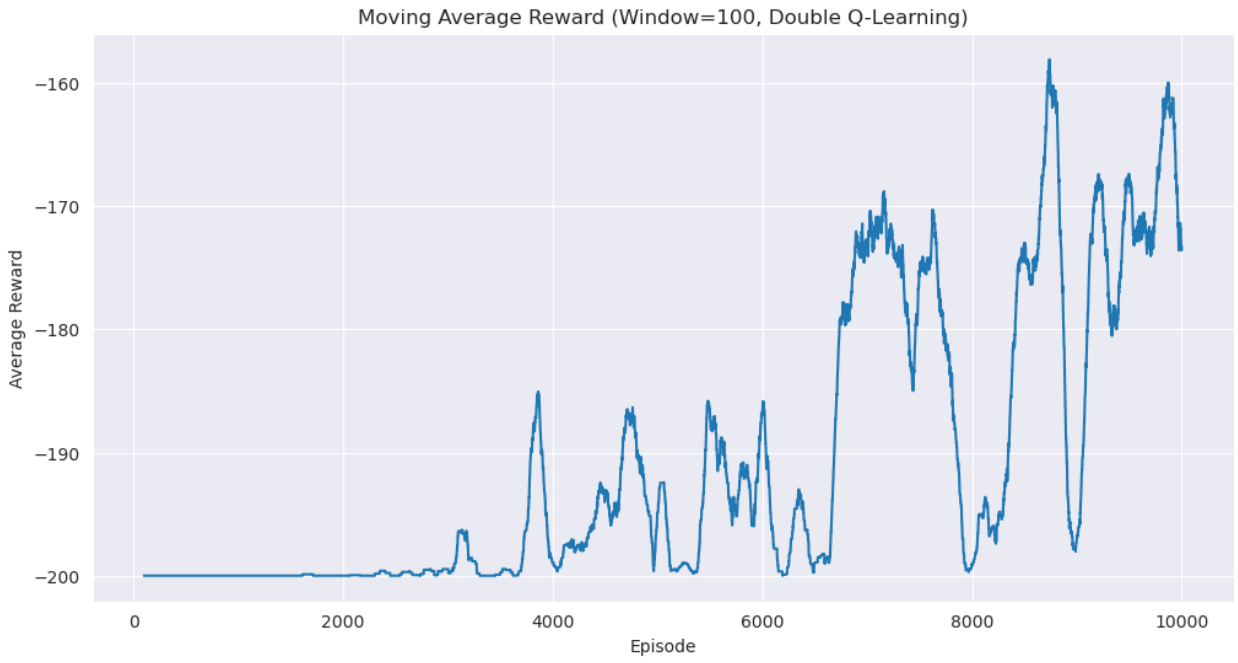
زمان آموزش برحسب ثانیه	۱۲۴,۱۵
پاداش (۱۰۰ اپیزود آخر)	-۱۷۳
درصد موفقیت (۱۰۰ اپیزود آخر)	۵۸٪
تعداد گام‌های طی‌شده (۱۰۰ اپیزود آخر)	۱۷۴



شکل ۷ – نمودار پاداش در طول یادگیری برای الگوریتم یادگیری Q دوگانه



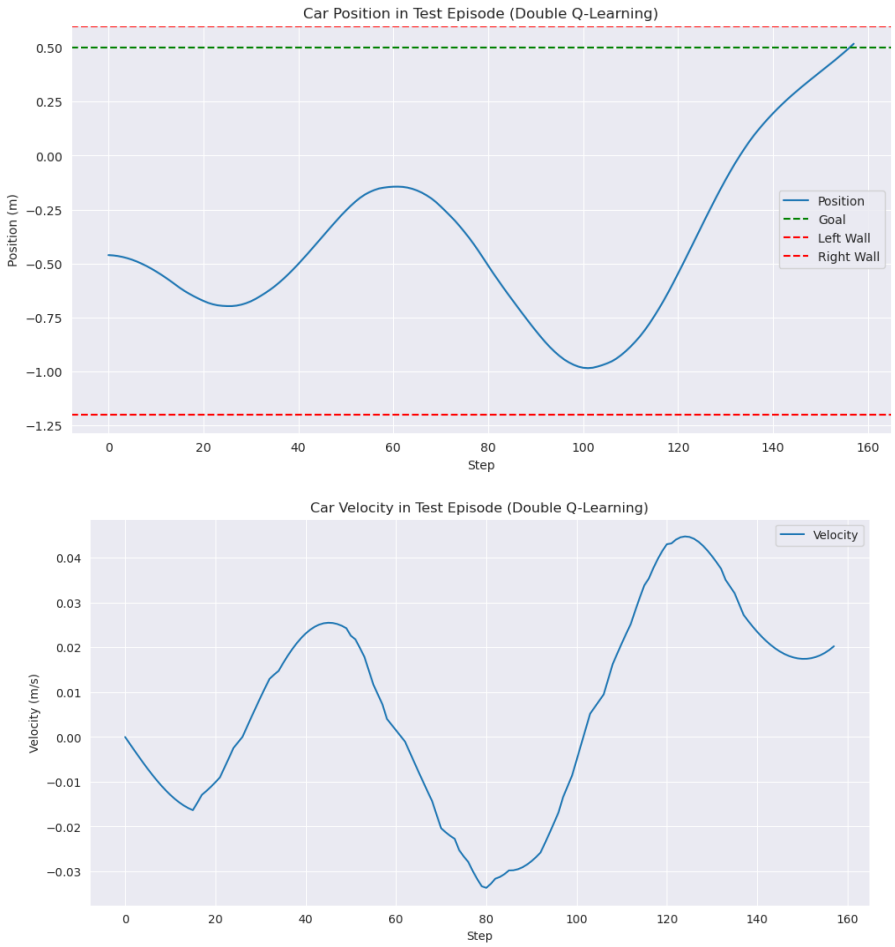
شکل ۸ – نمودار تعداد گام‌های طی‌شده در طول یادگیری برای الگوریتم یادگیری Q دوگانه



شکل ۹ – نمودار پنجره محرک پاداش به ازای هر مرحله در طول فرآیند یادگیری برای الگوریتم یادگیری Q دوگانه

همانطور که از نتایج مشخص است، این الگوریتم نتایج ضعیف‌تری نسبت به Q-Learning ساده ارائه می‌دهد. یک از دلایل آن با توجه به نمودار پنجره متحرک این باشد که این الگوریتم نیاز به تعداد اپیزودهای بیشتری برای یادگیری دارد.

عملکرد این الگوریتم در اپیزود تست Double Q-Learning قرار گرفته شده است و نشان می‌دهد این الگوریتم هم می‌تواند خودرو را به بالای تپه برساند اما به طور واضح سیاست آن در اتخاذ سرعت متفاوت از الگوریتم Q-Learning می‌باشد.



شکل ۱۰ – نمودار موقعیت و سرعت خودرو در اپیزود تست

۱-۴ الگوریتم DQN

DQN از یک شبکه عصبی پیش‌خور ساده برای تقریب تابع Q استفاده می‌کند و حالت‌ها را به مقادیر عمل نگاشت می‌کند. از آنجایی که فضای حالت کوچک است (موقعیت و سرعت، ۲ ورودی) و فضای عمل گسسته است (۳ عمل: بدون حرکت، چپ، راست)، به یک معماری مینیمال نیاز داریم تا از بیش‌برازش یا overfitting جلوگیری کنیم و در عین حال ظرفیت کافی برای یادگیری مقادیر Q را تضمین کنیم.

معماری شبکه عصبی استفاده‌شده بصورت زیر می‌باشد:

لایه ورودی: ۲ نورون (موقعیت، سرعت). برخلاف یادگیری Q، ما از مقادیر حالت پیوسته مستقیماً (نه گسسته) استفاده می‌کنیم، زیرا DQN می‌تواند ورودی‌های پیوسته را مدیریت کند.

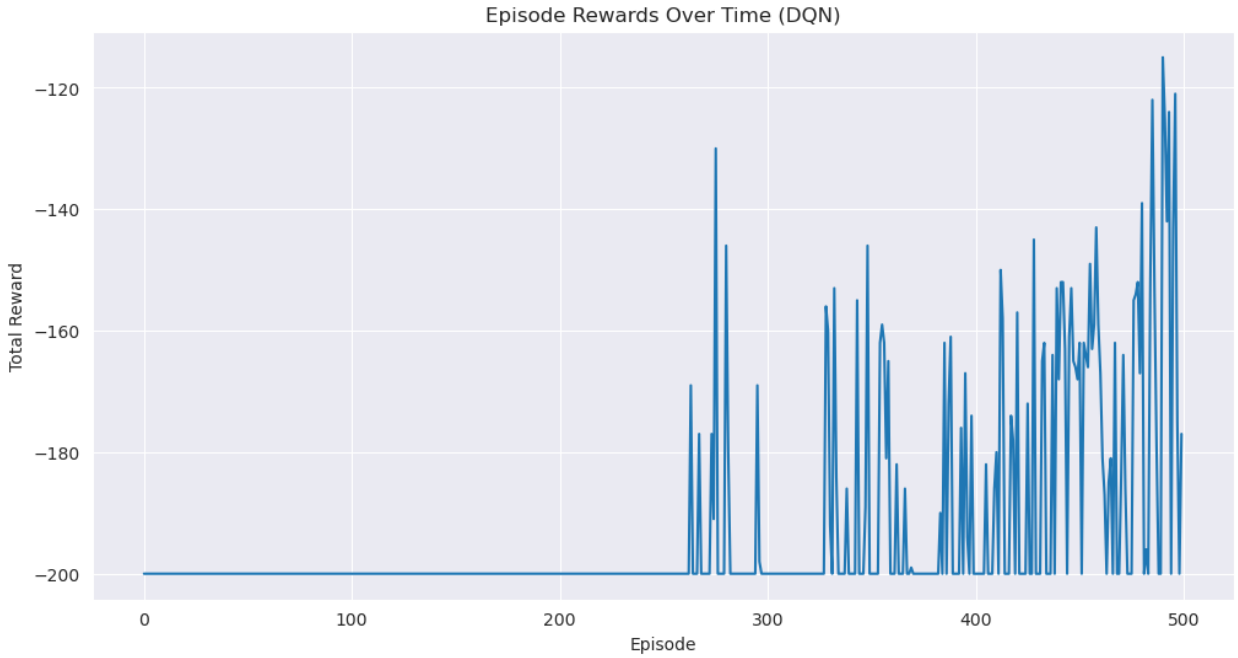
لایه‌های پنهان: دو لایه کاملاً متصل (متراکم) با ۶۴ نورون برای هر کدام. فعال‌سازی ReLU برای غیرخطی بودن

لایه خروجی: ۳ نورون (یکی برای هر عمل: بدون حرکت، چپ، راست)، مقادیر Q را خروجی می‌دهد.

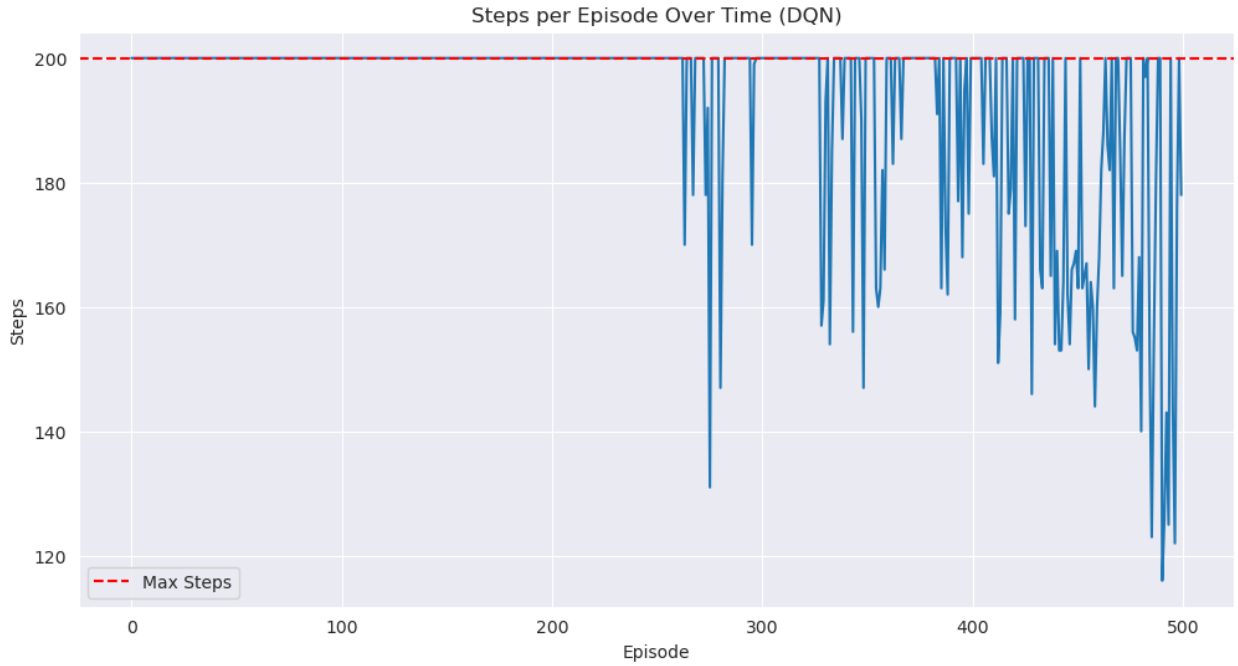
نتایج الگوریتم DQN برای مقادیر گسسته خروجی بصورت زیر می‌باشند:

زمان آموزش برحسب ثانیه	۲۹۰,۵۶
پاداش (۱۰۰ اپیزود آخر)	-۱۷۶
درصد موفقیت (۱۰۰ اپیزود آخر)	۵۹٪
تعداد گام‌های طی‌شده (۱۰۰ اپیزود آخر)	۱۷۹

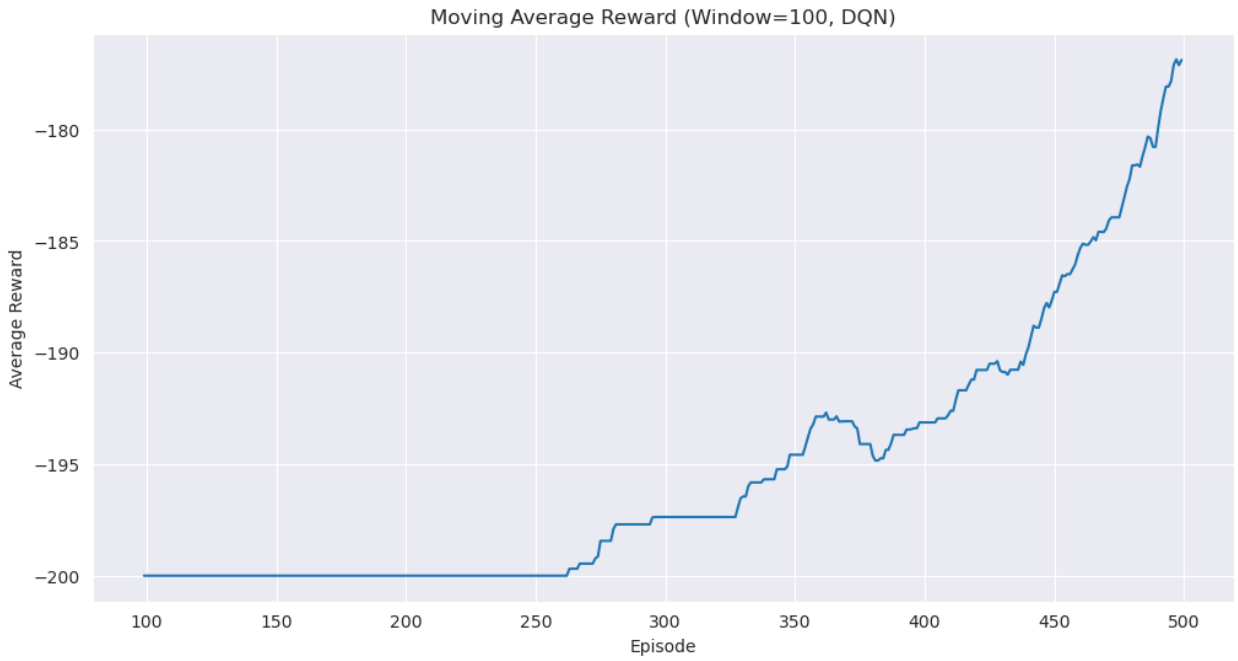
همانطور که مقایسه این مدل با دو مدل قبلی آشکار می‌سازد، نتایج DQN تقریبا مشابه Double Q-Learning بوده اما مدت زمان زمان آموزش آن تقریبا ۳ برابر می‌باشد. نکته مثبت الگوریتم DQN، امکان تغییر معماری شبکه می‌باشد. نتایج حاصل شده با یک معماری بسیار ساده و ابتدایی بدست آورده شده و با افزایش تعداد نورون‌ها و تعداد لایه‌ها می‌توان به نتایج بسیار بهتری نیز رسید. این امر باعث می‌شود در محیط‌های پیچیده‌تر که دارای فضای حالت بیشتری می‌باشند، DQN مطلوب‌تر از Q-Learning باشد.



شکل ۱۱ – نمودار پاداش در طول یادگیری برای الگوریتم DQN



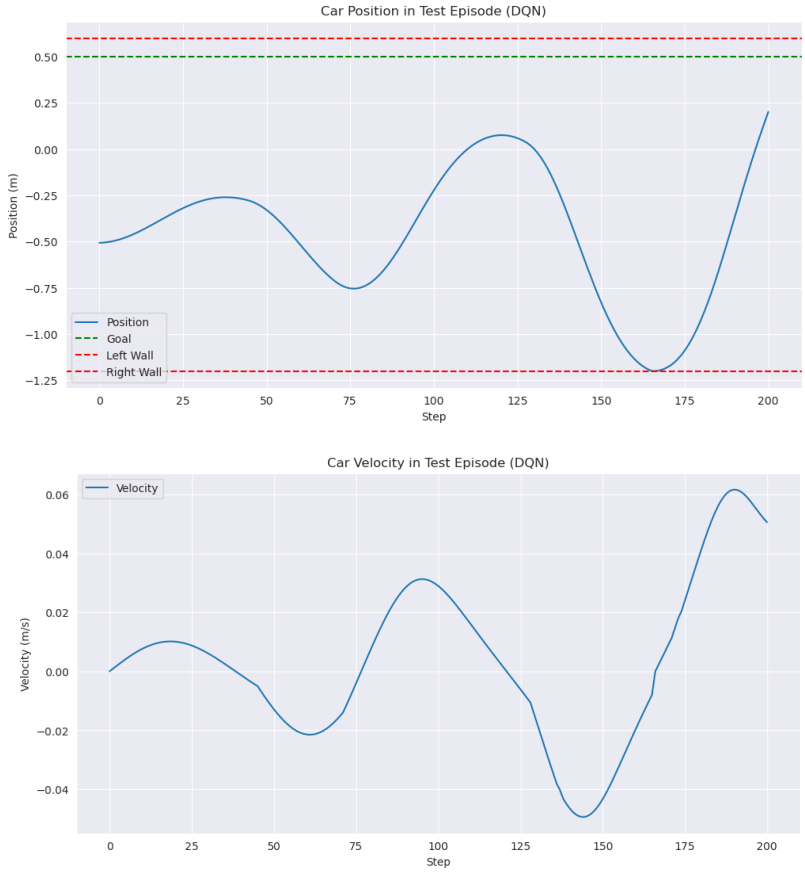
شکل ۱۲ – نمودار تعداد گام‌های طی‌شده در طول یادگیری برای الگوریتم DQN



شکل ۱۳ – نمودار پنجره محرک پاداش به ازای هر مرحله در طول فرآیند یادگیری برای الگوریتم DQN

شکل ۱۳ نشان می‌دهد که در انتهای ۵۰۰ اپیزود الگوریتم با شیب زیادی در حال یادگیری بوده و نیاز به اپیزودهای بیشتری برای یادگیری می‌باشد. اگر تعداد اپیزودهای یادگیری را بیشتر کنیم به نتایج بسیار بهتری برای این الگوریتم خواهیم رسید.

اپیزود تست این الگوریتم با نام DQN-Discrete در پوشه تست قرار گرفته شده و حاکی از این است که خودرو با موفقیت به پرچم می‌رسد.



شکل ۱۴ – نمودار موقعیت و سرعت خودرو در اپیزود تست

۵-۱ بررسی تاثیر هایپرپارامترها در الگوریتم DQN

برای بررسی اینکه چه پارامترهایی بر الگوریتم DQN تاثیر می‌گذارند سه نوع دیگر از ساختارهای متداول که در RL بررسی می‌شوند را لحاظ می‌کنیم:

۱- بهینه‌ساز RMSprop : در مقایسه با Adam، بهینه‌ساز RMSprop ساده‌تر بوده و تنها با میانگین‌گیری نمایی از مربع گرادیان‌ها نرخ یادگیری را برای هر پارامتر تنظیم می‌کند، در حالی که Adam علاوه بر این، از تخمین ممان اول (میانگین گرادیان) نیز بهره می‌گیرد. Adam معمولاً سریع‌تر همگرا می‌شود اما در محیط‌های نویزی مانند RL گاهی دچار نوسان می‌شود، در حالی که RMSprop به دلیل ماهیت ساده‌تر و تمرکز بر کنترل نوسان گرادیان‌ها اغلب پایداری بهتری ایجاد می‌کند

۲- تابع خطای Huber : تفاوت اصلی MSE و Huber در حساسیت به داده‌های پرت است؛ MSE همه خطاها را به صورت درجه دو جریمه می‌کند و بنابراین نسبت به نمونه‌هایی با خطای بزرگ بسیار حساس است، اما Huber در خطاهای کوچک مانند MSE رفتار کرده و در خطاهای بزرگ شبیه MAE عمل می‌کند، در نتیجه از انفجار گرادیان جلوگیری کرده و پایداری بیشتری در فرایند یادگیری فراهم می‌سازد.

۳ – انواع مختلف تابع پاداش : تا اینجا تابع پاداش استفاده‌شده ۱- به ازای هر گام که Agent به هدف نمی‌رسد بوده، اما می‌توان از تابع پاداش‌های متنوعی مانند گذاشتن پاداش برای رسیدن به هدف، تابع پاداش منفی به‌ازای هر چند گام که عامل به هدف نمی‌رسد استفاده کرد.

بدین منظور با ترکیب عوامل مختلف گفته‌شده از ۶ مدل مختلف استفاده می‌کنیم:

۱- بهینه‌ساز: Adam - تابع خطا: MSE - تابع پاداش: ۱- به ازای هر گام نرسیدن به هدف

۲- بهینه‌ساز: Adam - تابع خطا: MSE - تابع پاداش: ۱- به ازای هر ۳ گام متوالی نرسیدن به هدف

۳- بهینه‌ساز: Adam - تابع خطا: MSE - تابع پاداش: ۱- به ازای هر گام نرسیدن به هدف و ۱۰۰+ برای رسیدن به هدف

۴- بهینه‌ساز: RMSprop - تابع خطا: MSE - تابع پاداش: ۱- به ازای هر گام نرسیدن به هدف

۵- بهینه‌ساز: Adam - تابع خطا: Huber - تابع پاداش: ۱- به ازای هر گام نرسیدن به هدف

۶- بهینه‌ساز: RMSprop - تابع خطا: Huber - تابع پاداش: ۱- به ازای هر گام نرسیدن به هدف و ۱۰۰+ برای رسیدن به هدف

نتایج این ۶ مدل در جدول زیر آورده شده‌اند:

مدل	درصد موفقیت - درصد	تعداد گام طی-شده	زمان آموزش مدل - ثانیه
۱	۶۷	۱۶۷	۳۲۱
۲	۹۹	۹۶	۲۶۱
۳	۹۸	۸۲	۲۲۲
۴	۰	۲۰۰	۳۰۲
۵	۰	۲۰۰	۳۳۶
۶	۰	۲۰۰	۳۰۱

نتایج آزمایش شش مدل مختلف DQN روی محیط MountainCar نشان می‌دهد که تغییر در تابع پاداش، به‌مراتب اثرگذارتر از تغییر در بهینه‌ساز یا تابع خطا بوده است. در مدل پایه (مدل ۱) با پاداش ثابت -۱ در هر گام، یادگیری بسیار کند بوده و در نهایت تنها بخشی از اپیزودها موفقیت‌آمیز شدند. در مدل ۲ که پاداش منفی تنها هر سه گام اعمال شد، عامل فشار کمتری برای سرعت بهینه‌سازی داشت و این باعث شد عامل به جای تمرکز بر کاهش فوری گام‌ها، فرصت بیشتری برای کشف راهبردهای مؤثر داشته باشد. نتیجه آن افزایش چشمگیر نرخ موفقیت (٪۹۹) و کاهش میانگین گام‌ها بود.

مدل ۳ با اضافه کردن پاداش مثبت بزرگ (+۱۰۰) در زمان رسیدن به هدف، توانست یادگیری جهت‌دارتری داشته باشد؛ چراکه عامل به‌وضوح سیگنال «اتمام موفقیت‌آمیز» دریافت می‌کرد. همین امر باعث شد میانگین پاداش مثبت شود و نرخ موفقیت نزدیک به ٪۹۸ باشد. در عوض، نوسان بیشتری در پاداش‌ها دیده شد که ناشی از شدت تفاوت میان اپیزودهای موفق و ناموفق بود.

مدل ۴ که تنها بهینه‌ساز را از Adam به RMSprop تغییر داد، هیچ یادگیری معناداری نشان نداد. علت را می‌توان در حساسیت RMSprop به نرخ یادگیری و ناپایداری گرادیان‌ها در محیط‌های با پاداش پراکنده دانست Adam . به دلیل داشتن مکانیسم مومنتوم و میانگین‌گیری تطبیقی از گرادیان‌ها، برای چنین محیط‌هایی پایدارتر عمل می‌کند.

مدل ۵ نیز با جایگزینی Huber loss به جایMSE ، تفاوتی ایجاد نکرد. دلیل این موضوع آن است که در این محیط، محدوده پاداش و خطاها به‌طور ذاتی کوچک است و مزیت) Huber کاهش اثر مقادیر پرت) کمتر بروز می‌یابد.

در نهایت، مدل ۶ که ترکیبی از RMSprop و Huber با پاداش +۱۰۰ بود، نیز ناکام ماند و نشان داد که حتی وجود پاداش قوی مثبت هم نمی‌تواند ضعف بهینه‌ساز و عدم پایداری ناشی از آن را جبران کند.

به‌طور کلی می‌توان نتیجه گرفت که در این مسئله، **ساختار تابع پاداش** تعیین‌کننده‌ترین عامل در موفقیت است و تغییرات در بهینه‌ساز و تابع خطا تنها در شرایطی خاص می‌توانند مؤثر باشند .Adam همراه با طراحی دقیق پاداش، بهترین ترکیب را برای رسیدن به عملکرد پایدار و نرخ موفقیت بالا فراهم کرده است.

۲. محیط با اقدام‌های پیوسته

برای حالت دوم محیط mountaincar نیاز است تا کلاسی جدید تعریف شود. تفاوت اساسی این کلاس با کلاس قبلی در تعریف پارامترهای ورودی و خروجی بوده که اینبار بجای مقادیر گسسته، دارای مقادیر پیوسته در بازه مورد نظر می‌باشند.

۲-۱ الگوریتم DQN

تفاوت این الگوریتم با الگوریتم DQN محیط گسسته، تنها در تعداد نورون‌های خروجی می‌باشد که بجای ۳ حالت گسسته مختلف، بازه ۱- تا ۱+ آن به ۱۰ قسمت تقسیم‌شده است.

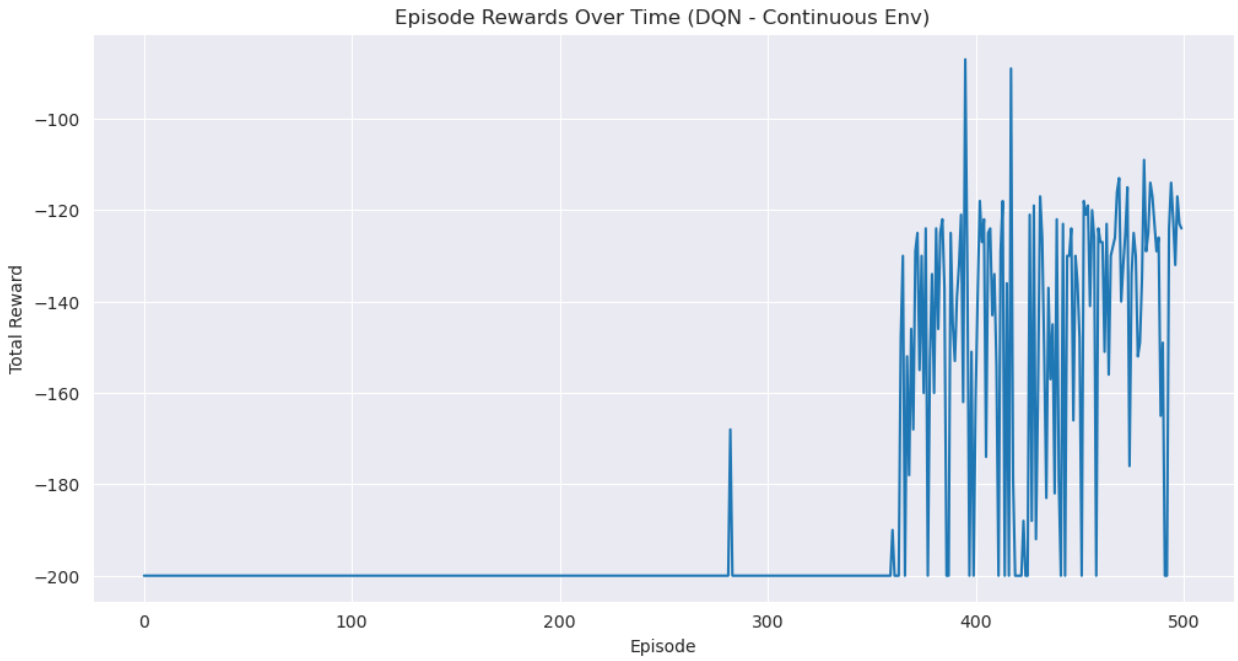
نتایج این مدل به‌شرح زیر می‌باشد:

زمان آموزش برحسب ثانیه	۱۴۱,۷۳
پاداش (۱۰۰ اپیزود آخر)	-۱۴۵
درصد موفقیت (۱۰۰ اپیزود آخر)	۸۵٪
تعداد گام‌های طی‌شده (۱۰۰ اپیزود آخر)	۱۴۶

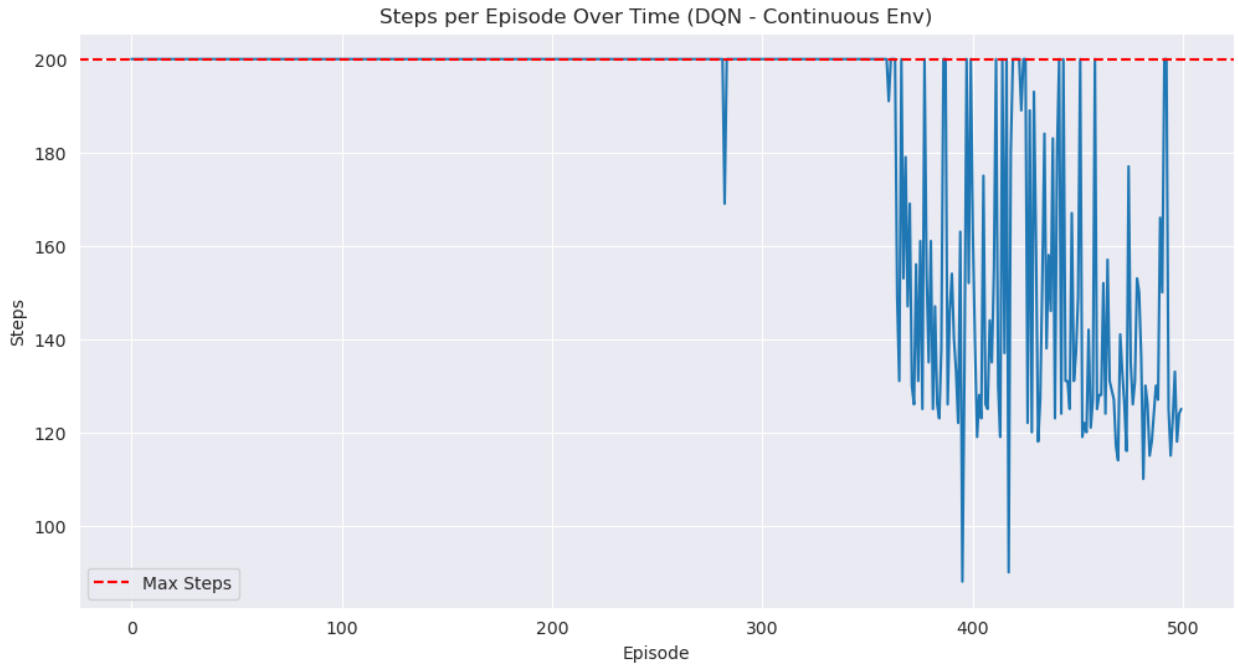
همانطور که از مقایسه نتایج DQN محیط‌های گسسته و پیوسته مشخص است، افزایش تعداد حالت‌های خروجی از ۳ به ۱۰، درصد موفقیت مدل را حدود ۲۵ درصد افزایش داده که نشان می‌دهد اگر عامل بتواند اقدام‌های خود را از فضای اقدام بزرگتری انتخاب کند و انعطاف‌پذیری بیشتری داشته باشد، نتایج بسیار بهتری خواهد گرفت.

قابل ذکر است که الگوریتم DQN در عمل برای محیط‌های گسسته مورد استفاده قرار گرفته و برای اینکه برای محیط کاملا پیوسته نیز قابل پیاده‌سازی باشد نیاز است که تعداد بسیار زیادی (بی‌نهایت) نورون در لایه خروجی قرار دهیم که غیرممکن می‌باشد. محیط الگوریتم DQN استفاده شده در این بخش عملا پیوسته نبوده و تنها تعداد اقدام‌های آن افزایش یافته‌است.

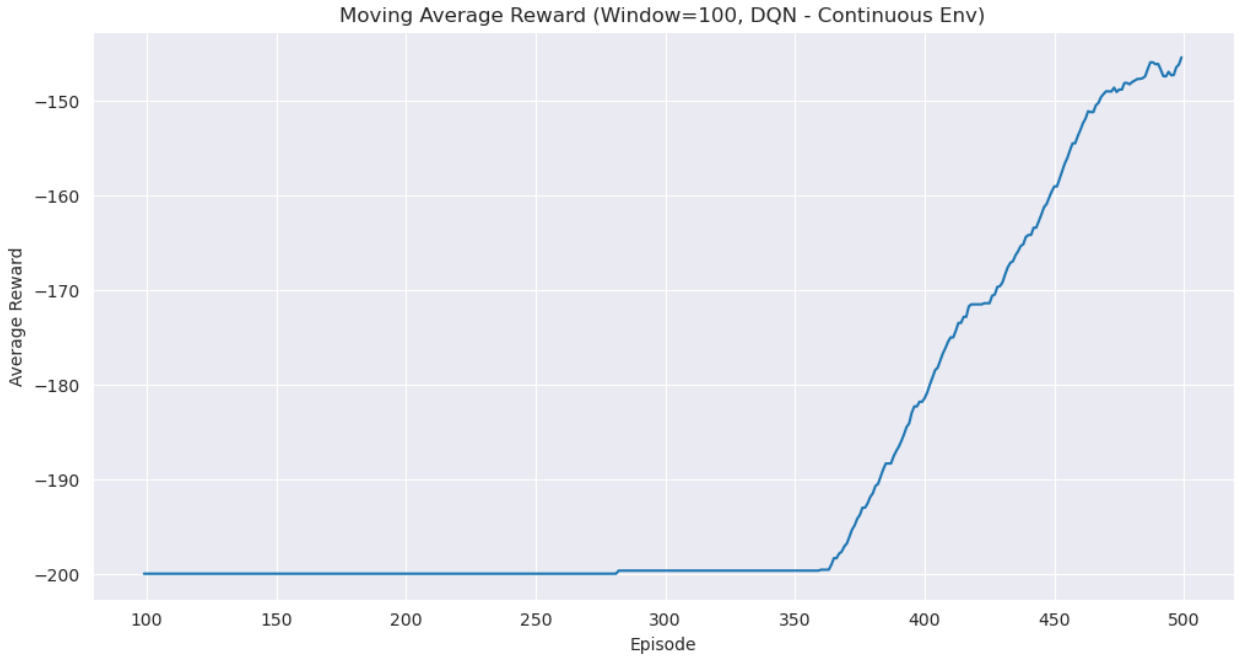
یکی از نکات قابل توجه که در آزمایش انواع این الگوریتم مشخص شد این بود که استانداردسازی حالت‌ها (قرار دادن بین ۱- تا ۱+) نقش قابل توجهی در بهبود الگوریتم دارد.



شکل ۱۵ – نمودار پاداش در طول یادگیری برای الگوریتم DQN با اقدام‌های پیوسته

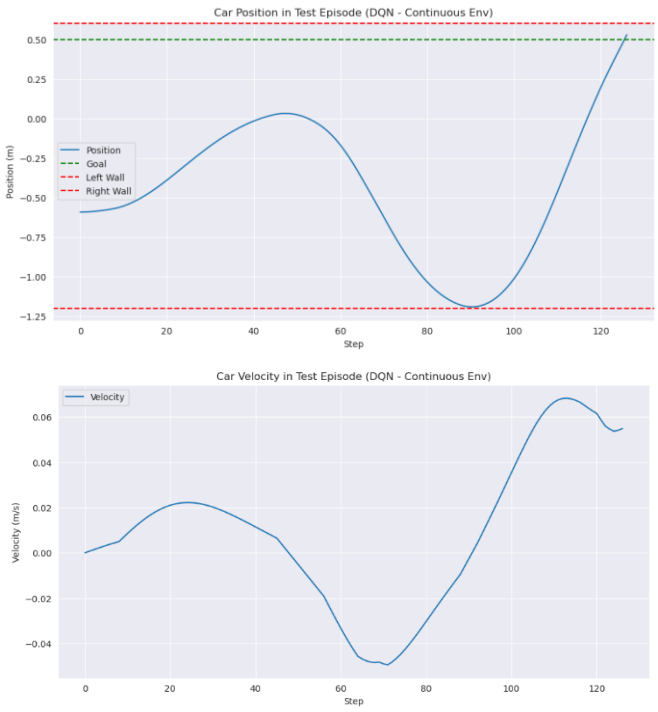


شکل ۱۶ – نمودار تعداد گام‌های طی‌شده در طول یادگیری برای الگوریتم DQN با اقدام‌های پیوسته



شکل ۱۷ – نمودار پنجره محرک پاداش به ازای هر مرحله در طول فرآیند یادگیری برای الگوریتم DQN با اقدام‌های پیوسته

اپیزود تست این الگوریتم با نام DQN-Continues قرار داده‌شده است.



شکل ۱۸ – نمودار موقعیت و سرعت خودرو در اپیزود تست

۲-۲ الگوریتم DDPG

برای این الگوریتم معماری‌ها و پارامترهای مختلفی امتحان شده‌است. هر معماری را ذکر کرده و نتایج آن را بررسی و تفسیر می‌کنیم.

در تلاش اول معماری دو شبکه بازیگر و منتقد و همچنین پارامترها شبیه به گزارش داده‌شده تنظیم شده است و تنها تفاوت، کم شدن تعداد نورون‌ها در شبکه است.

شبکهٔ Actor

- یک لایه خطی با اندازه ۱۲۸ با تابع ReLU
- یک لایه خطی با اندازه ۶۴ با تابع ReLU
- یک لایه خطی با اندازه خروجی شبکه با تابع Tanh
- تابع فوروارد مخصوص Actor

شبکهٔ Critic

- یک لایه خطی با اندازه ۱۲۸ با تابع ReLU
- یک لایه خطی برای سر هم کردن خروجی لایه قبلی و خروجی Actor
- یک لایه خطی با اندازه ۱ با تابع ReLU
- تابع فوروارد مخصوص Critic

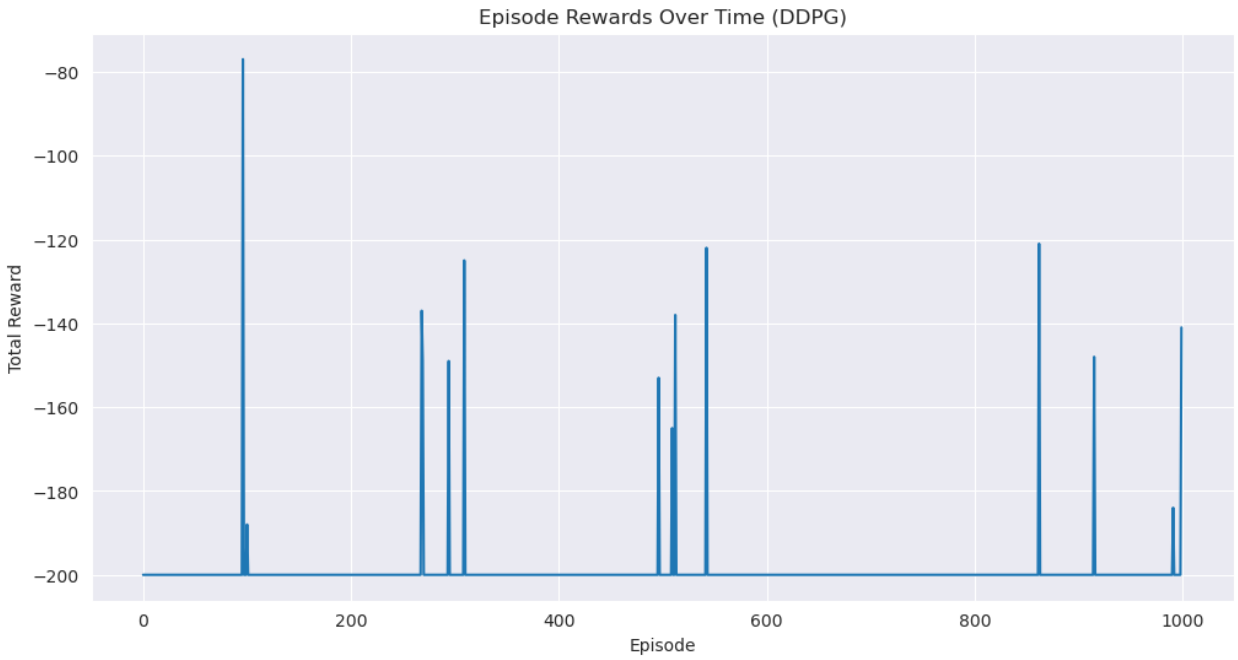
تابع نویز Ornstein-Uhlenbeck با mu=0.15 و sigma=0.2

- ضریب کاهش: gamma=0.99
- ظرفیت حافظه: buffer_capacity=100000
- تعداد مراحل: n_episodes=1000
- حداکثر تعداد تلاشها در هر مرحله: max_steps=200
- استفاده از تابع Adam به عنوان بهینه‌از با نرخ یادگیری ۰,۰۰۱
- استفاده از تابع خطای MSE

• ضریب تاو در soft update : Tau=0.001

نتایج آزمایش اول به شکل زیر می‌باشد:

زمان آموزش برحسب ثانیه	۸۶۸,۵۵
پاداش (۱۰۰ اپیزود آخر)	-۱۹۸
درصد موفقیت (۱۰۰ اپیزود آخر)	۳٪
تعداد گام‌های طی‌شده (۱۰۰ اپیزود آخر)	۱۹۸



شکل ۱۹ – نمودار پاداش در طول یادگیری برای الگوریتم DDPG – تلاش اول

همانطور که از شکل ۱۹ مشخص است، یادگیری مدل بسیار نویزی بوده و در برخی اپیزودهای به خصوص یادگیری انجام شده و دوباره به حالت اولیه برمی‌گردد. نتایج نیز نشان می‌دهد علی‌رغم زمان آموزش بسیار زیاد، خودرو تنها در ۳ درصد مواقع به بالای تپه می‌رسد.

فیلم عملکرد این مدل با نام DDPG – Test 1 نیز نشان می‌دهد که رفتار عامل نویزی بوده و سیاست یادگیری‌شده منجر به موفقیت نمی‌شود. اگر با دقت بیشتری به تست نگاه کنیم متوجه می‌شویم که عامل سیاست حرکت به سمت راست را به طور کامل نیاموخته است.

از دلایل احتمالی این روند می‌توان به زیاد بودن نویز، عدم پایداری در روند یادگیری و پیچیدگی زیاد مدل و در نتیجه بیش‌برازش آن اشاره کرد.

در مدل دوم DDPG پارامترهای زیر را تغییر می‌دهیم:

- کاهش تعداد نورون‌ها از ۱۲۸ و ۶۴ به ۶۴ و ۳۲ جهت ساده‌سازی مدل

- کاهش نرخ یادگیری شبکه منتقد (Learning rate) جهت پایدارسازی

- تغییر تابع پاداش جهت تشویق عامل برای اتخاذ سیاست‌هایی که خودرو را به سمت راست ببرد

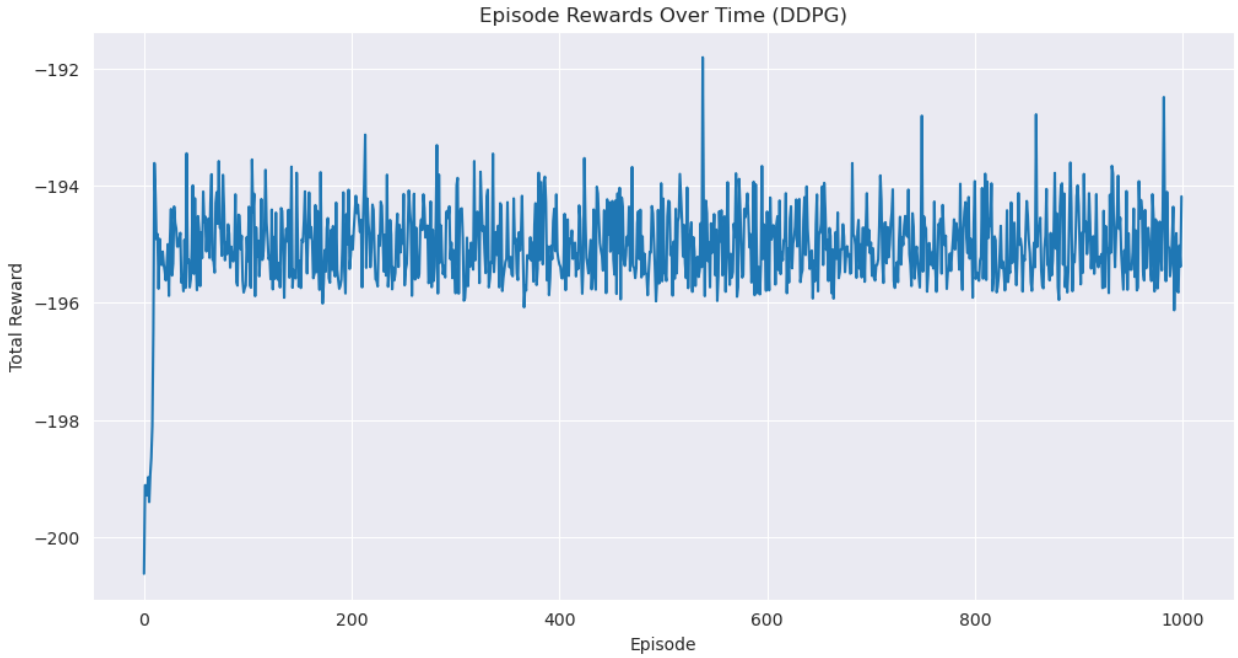
- افزایش ضریب تاو برای آپدیت سریعتر

- کاهش نویز Ornstein-Uhlenbeck

- افزایش Batch size برای پایدارسازی گرادیان‌ها

با تغییرات ذکر شده، نتیجه مدل دوم بصورت زیر خواهد بود:

زمان آموزش برحسب ثانیه	۸۳۶,۴۹
پاداش (۱۰۰ اپیزود آخر)	-۱۹۴
درصد موفقیت (۱۰۰ اپیزود آخر)	۰٪
تعداد گام‌های طی‌شده (۱۰۰ اپیزود آخر)	۲۰۰



شکل ۲۰ – نمودار پاداش در طول یادگیری برای الگوریتم DDPG – تلاش دوم

همینطور که مشخص است، نتایج این مدل نیز موفقیت آمیز نیست و اپیزود تست DDPG – test 2 نشان می‌دهد که به دلیل سیاست پاداش‌دهی حرکت به سمت راست، عامل تمایل زیادی به حرکت به این سمت داشته و حرکت به سمت چپ برای ایجاد momentum را اتخاذ نمی‌کند.

برای تلاش بعدی تغییرات زیر را اعمال می‌کنیم:

- تغییر تابع پاداش جهت تشویق برای ایجاد مومنتم و همچنین ۱۰۰+ جهت رسیدن به هدف

- ایجاد نرخ کاهش نویز (noise with decay)، بدین ترتیب با گذشت زمان مقدار نویز کمتر می‌شود

- تنظیم پارامترهای بهینه‌ساز Adam جهت بهبود پایداری

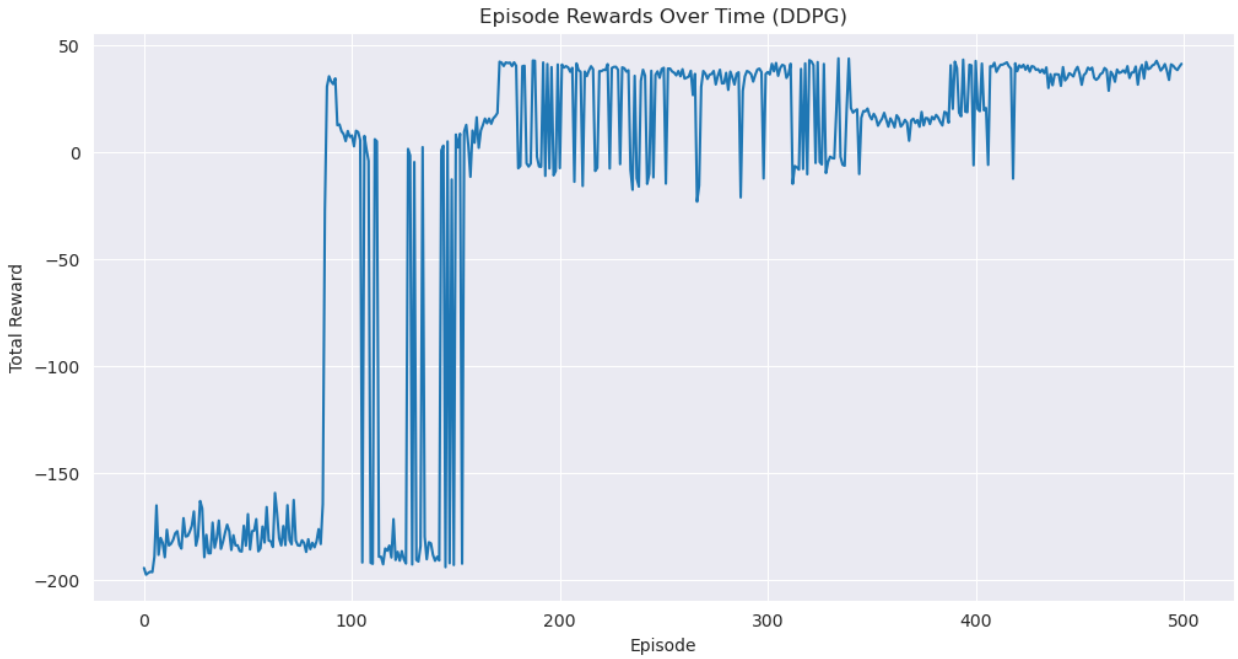
- کاهش تعداد اپیزودهای یادگیری به ۵۰۰ اپیزود

- کاهش نرخ Exploration

نتایج این مدل به شرح زیر می‌باشد:

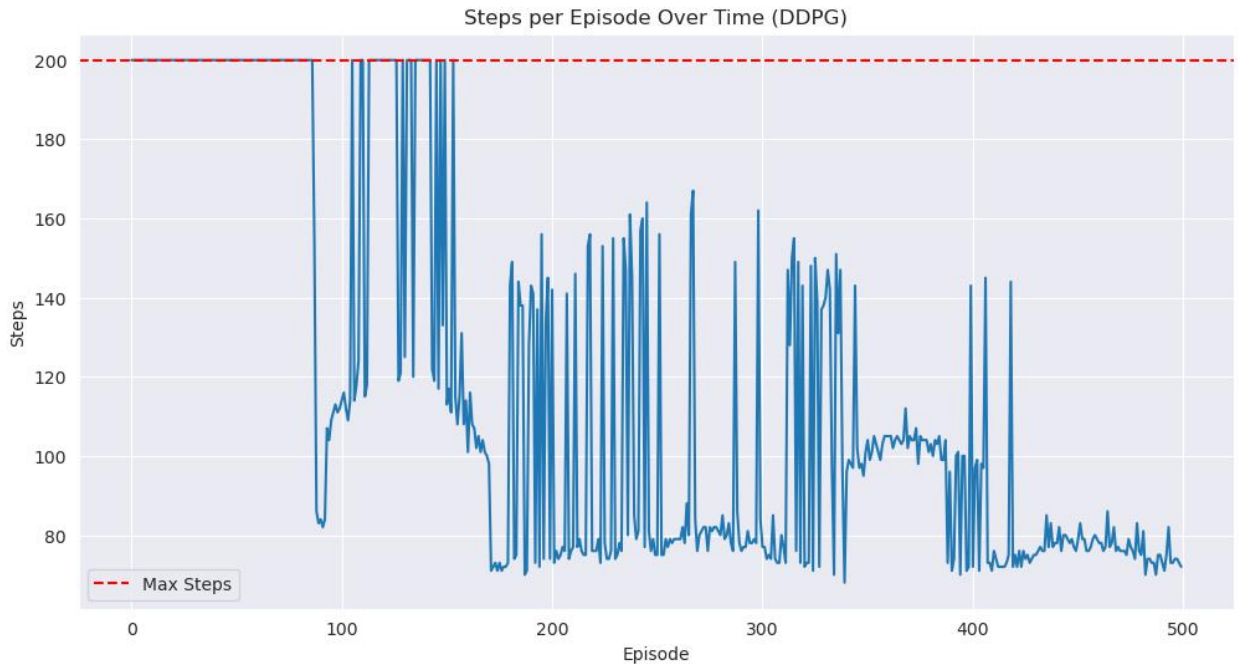
زمان آموزش برحسب ثانیه	۳۸۸,۴۵
پاداش (۱۰۰ اپیزود آخر)	۳۶
درصد موفقیت (۱۰۰ اپیزود آخر)	۱۰۰٪
تعداد گام‌های طی‌شده (۱۰۰ اپیزود آخر)	۷۸

همانطور که مشخص است، تغییرات اعمال‌شده کاملا اثربخش بوده و عامل در تعداد گام‌های کمی در ۱۰۰ درصد مواقع به هدف می‌رسد. همچنین قابل ذکر است که با ساده‌سازی مدل، زمان آموزش نیز تقریبا نصف شده‌است.

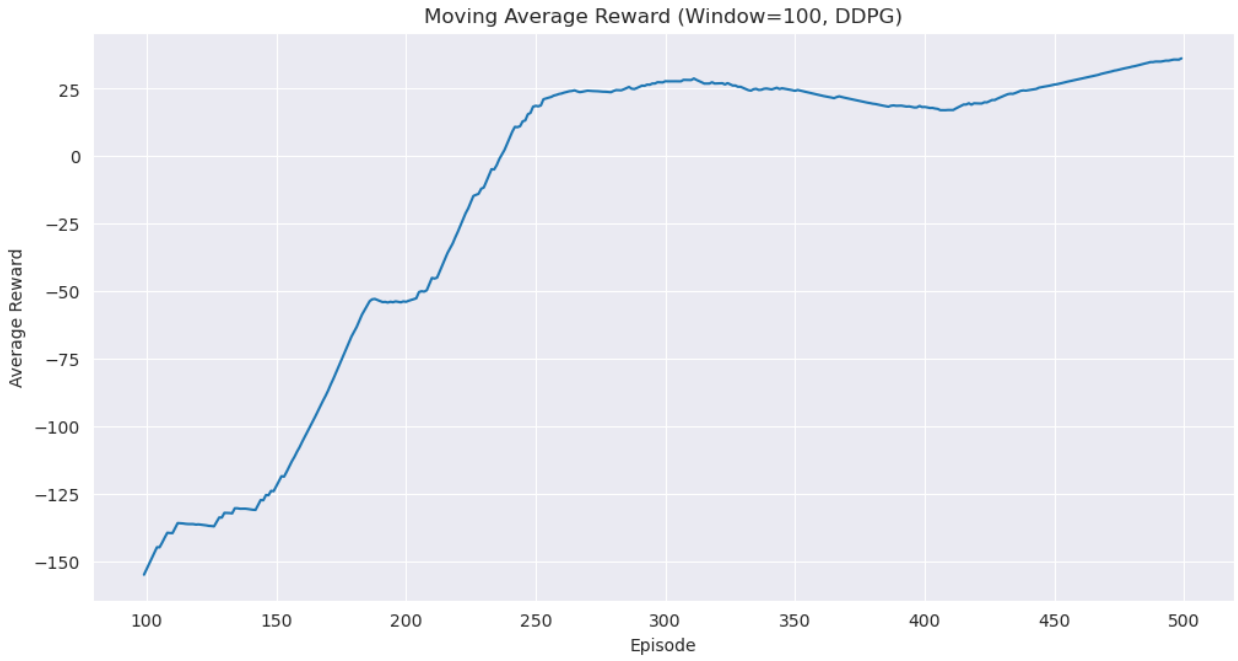


شکل ۲۱ – نمودار پاداش در طول یادگیری برای الگوریتم DDPG – تلاش سوم

می‌توان به این نتیجه رسید که تفاوت اصلی این مدل و مدل‌های قبلی در تنظیم تابع پاداش می‌باشد. این امر اهمیت تنظیم تابع پاداش مناسب را در یادگیری تقویتی آشکار می‌سازد.

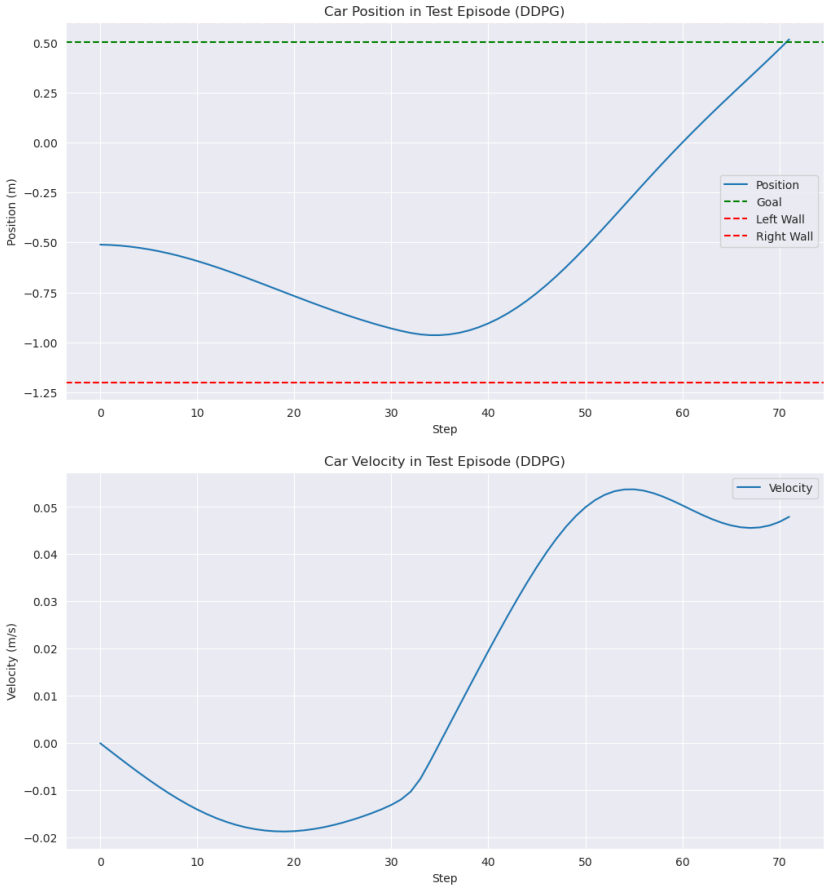


شکل ۲۲ – نمودار تعداد گام‌های طی‌شده در طول یادگیری برای DDPG – تلاش سوم



شکل ۲۳ - نمودار پنجره محرک پاداش به ازای هر مرحله در طول فرآیند یادگیری برای DDPG – تلاش سوم

همچنین نتایج اپیزود تست با نام DDPG – test 3 نشان می‌دهد که با تعداد گام بسیار کم و در سریعترین زمان عامل به پرجم می‌رسد.



شکل ۲۴ – نمودار موقعیت و سرعت خودرو در اپیزود تست

۳. مقایسه مدل‌ها و نتیجه‌گیری

مقایسه مدل‌ها از ۳ منظر درصد موفقیت (در ۱۰۰ اپیزود آخر یادگیری)، تعداد گام طی‌شده (در ۱۰۰ اپیزود آخر یادگیری) و زمان آموزش مدل انجام شده‌است.

	درصد موفقیت - درصد	تعداد گام طی-شده	زمان آموزش مدل - ثانیه
Q-Learning	100	149	92
Double Q-Learning	58	174	124
DQN (Discrete)	59	179	290
DQN (Discrete) Optimized	99	96	261
DQN (continues)	85	146	141
DDPG Optimized	100	78	388

طبعا چون تابع پاداش‌های این مدل‌ها متفاوت می‌باشند، مقایسه مقادیر پاداش آن‌ها انجام نمی‌شود.

همانطور که مشاهده می‌شود هر دو مدل Q – Learning و DDPG موفق به عملکرد صددرصدی در اپیزودهای تست شده‌اند. تفاوت اصلی این دو الگوریتم در آن است که الگوریتم DDPG بسیار سریعتر و در تعداد گام‌های کمتری به هدف می‌رسد که نشان می‌دهد سیاست عامل هوشمندانه‌تر است. از طرف دیگر زمان آموزش مدل در Q-Learning با توجه عدم نیاز به شبکه عمیق، بسیار سریعتر است.

در رده‌های بعدی الگوریتم DQN بهینه‌شده، الگوریتم DQN محیط پیوسته و در نهایت Q – Learning دوگانه است. در نهایت می‌توان گفت که مسئله موردبحث به علت فضای حالت کوچک از جمله مسائل ساده به‌شمار رفته و الگوریتم ساده‌ای مانند Q – Learning نیز توانایی حل آن را دارد. الگوریتم DDPG به علت داشتن دو شبکه جداگانه که اقدام را تعیین می‌کند و ارزش آن را تخمین می‌زند برای حل این مسئله کارآمد بوده و در مسائل پیچیده‌تر نیز، قابلیت یادگیری سیاست بهینه و سریع را دارا می‌باشد.

نکته بسیار مهم که از آزمایش مدل‌ها با پارامترهای مختلف بدست آمد، اهمیت پیاده‌سازی تابع پاداش در مسائل یادگیری تقویتی است. بدین صورت که تغییر تابع پاداش تاثیر بسزایی در عملکرد مدل‌ها داشته و گذاشتن تشویق برای رسیدن مدل به هدف می‌تواند تعداد گام‌های رسیدن عامل به پرچم را کاهش دهد.